

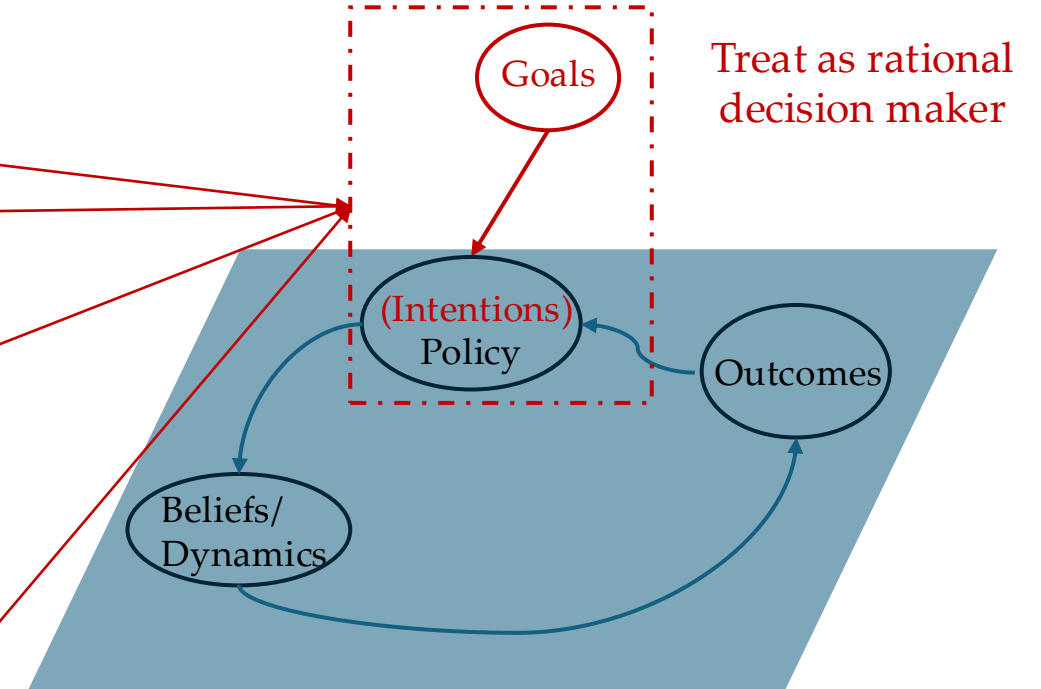
Inverse Reinforcement Learning for Optimal Control Systems and Games

Bosen Lian, PhD
Assistant Professor, ECE, Auburn University

We interact with the world (agent) by understanding its goals



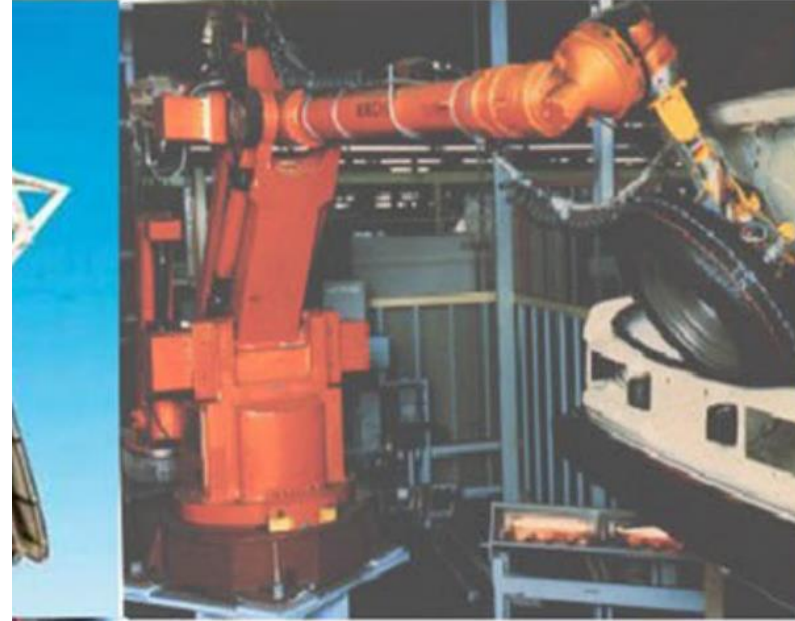
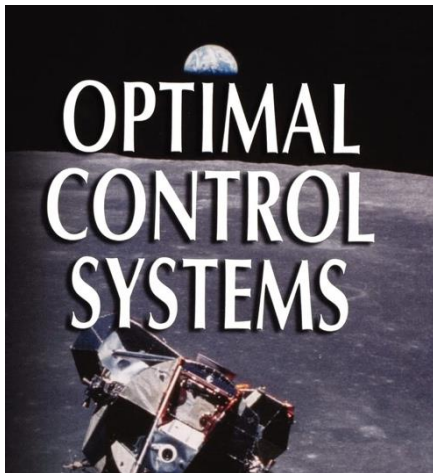
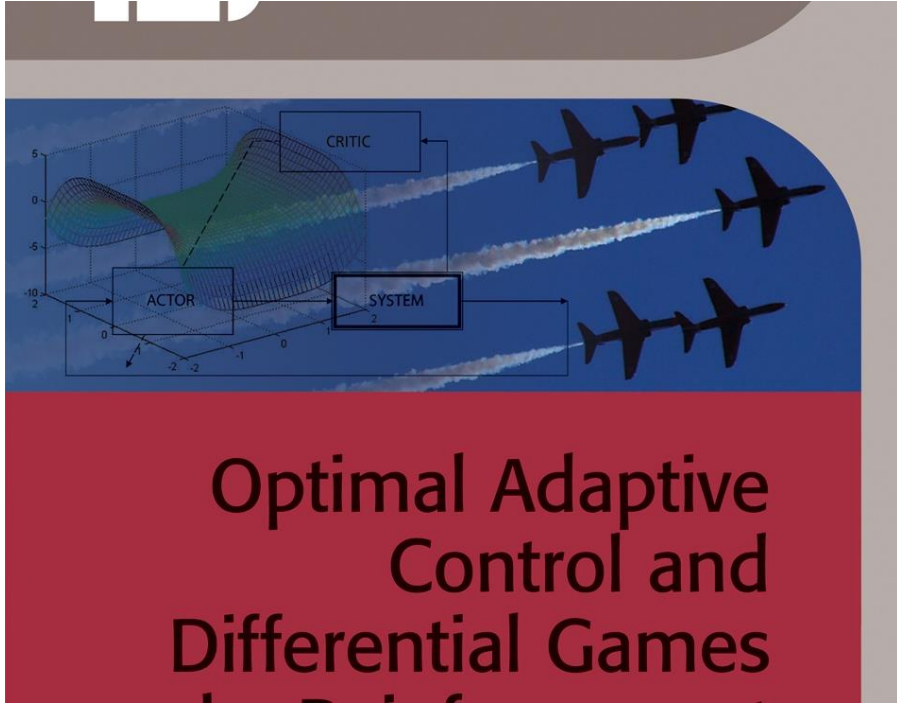
Better Explain and Predict



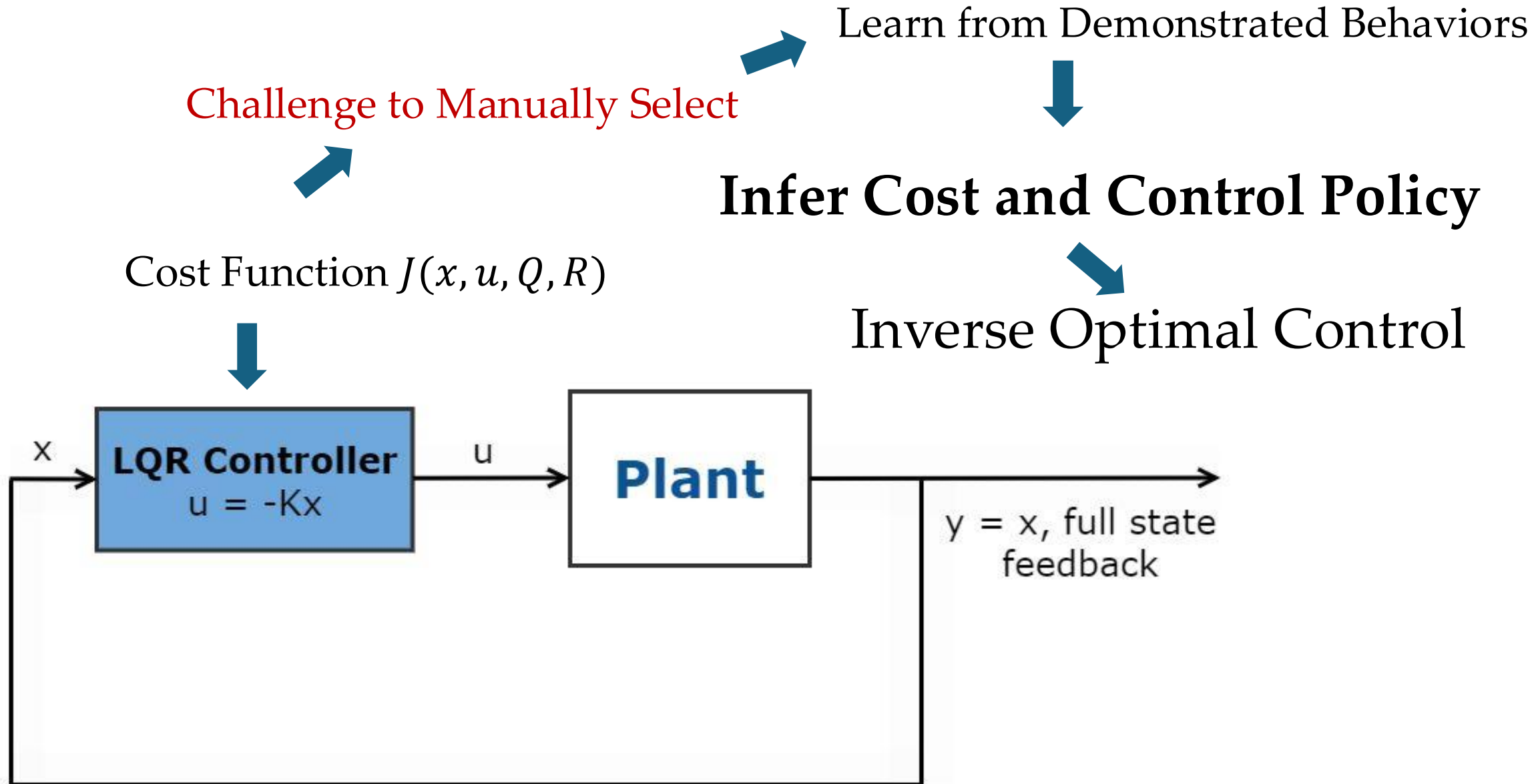
Theory of Mind

Infer Goals and Intention Policy

We design autonomous systems using optimal control



Optimal Control VS Inverse Optimal Control



To infer goal/cost and optimal control policy

Existing Studies and Challenges

Inverse RL:

Discrete MDP
Decision making
Computation load
Requiring low-level
control

Inverse Optimal Control:

Continuous states &
controls
Known models
Constant reward

However

Autonomous Control systems (humans, robots, vehicles):

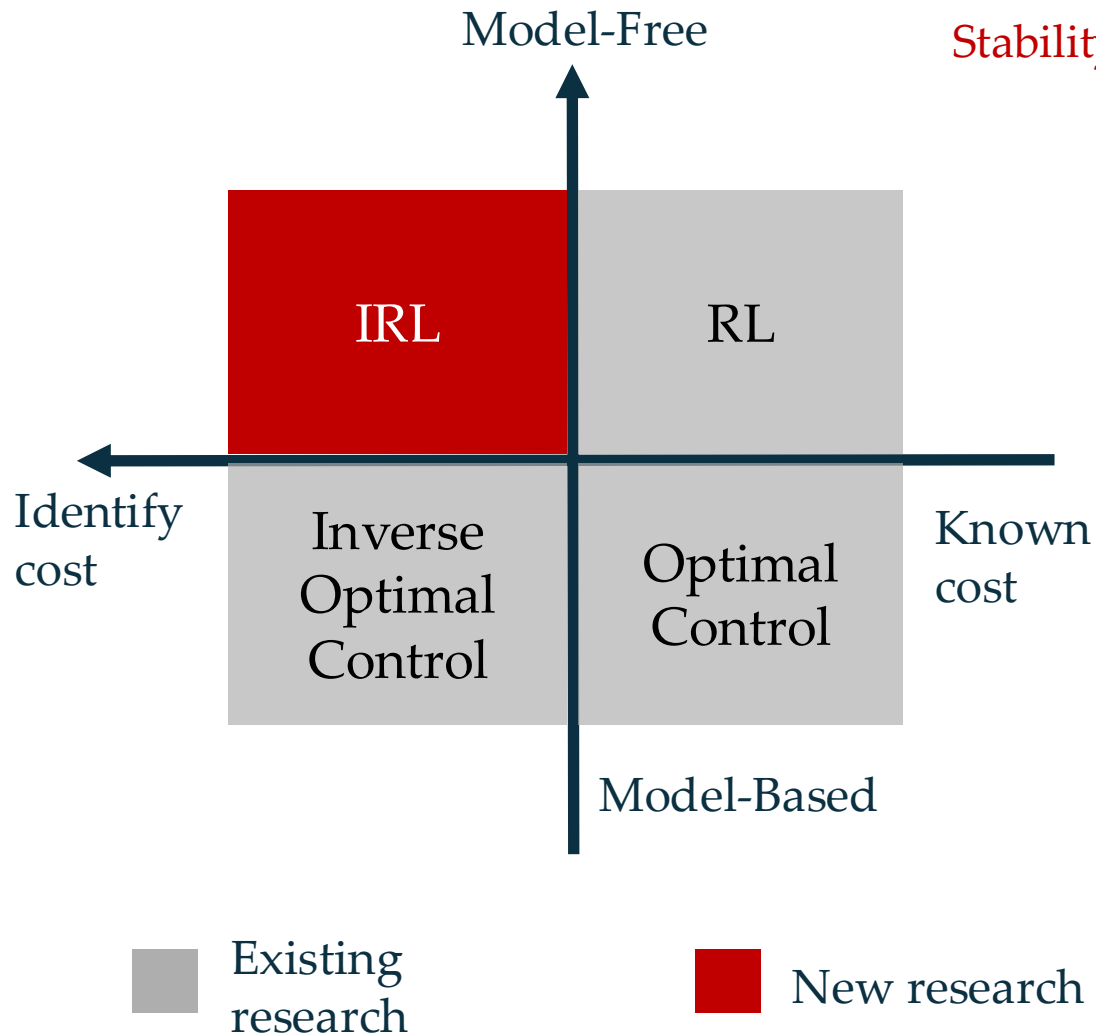
Model uncertainties
Requiring stability
Noisy observation
Data inefficiency



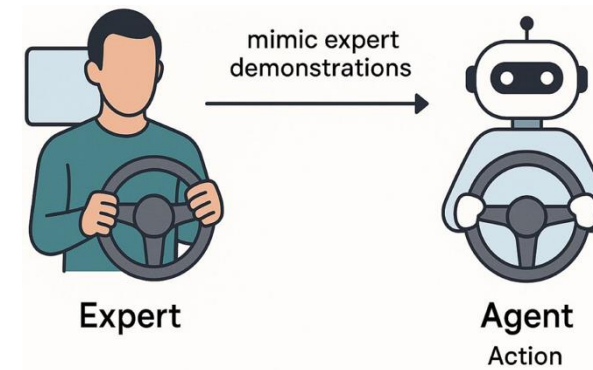
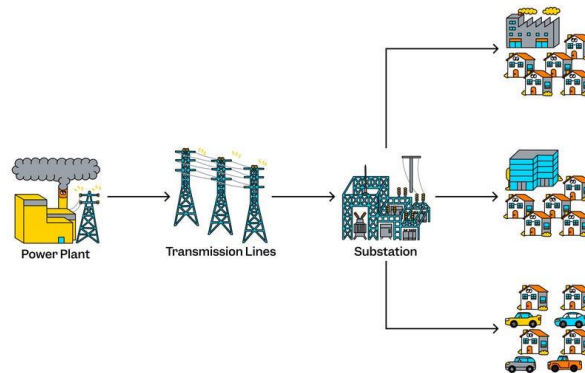
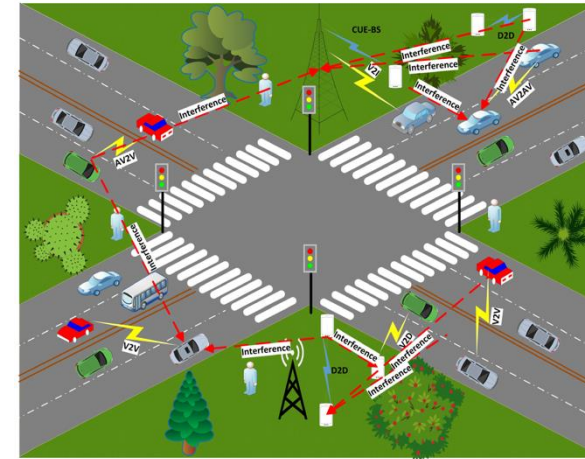
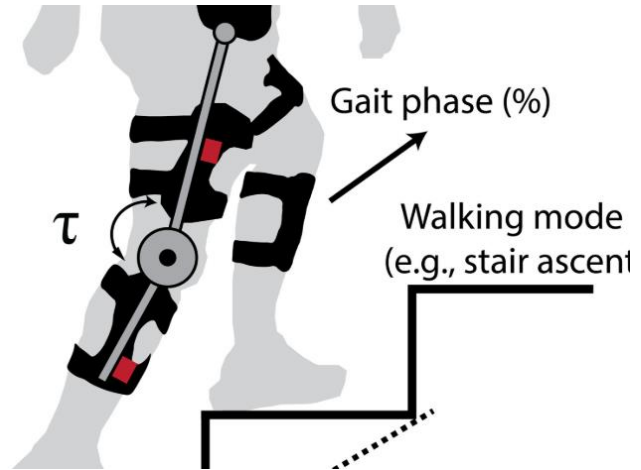
Need Model-free Inverse RL
in control systems

Inverse RL in Control Systems

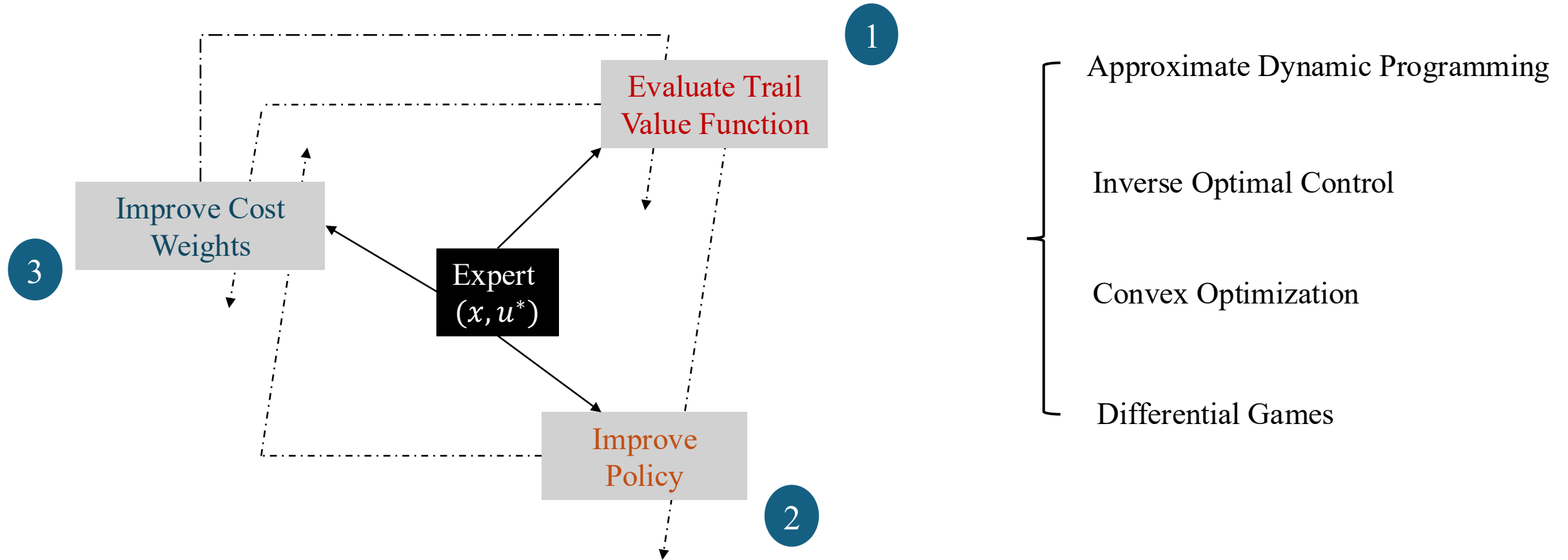
- We pioneer a new discipline of inverse RL in control theory



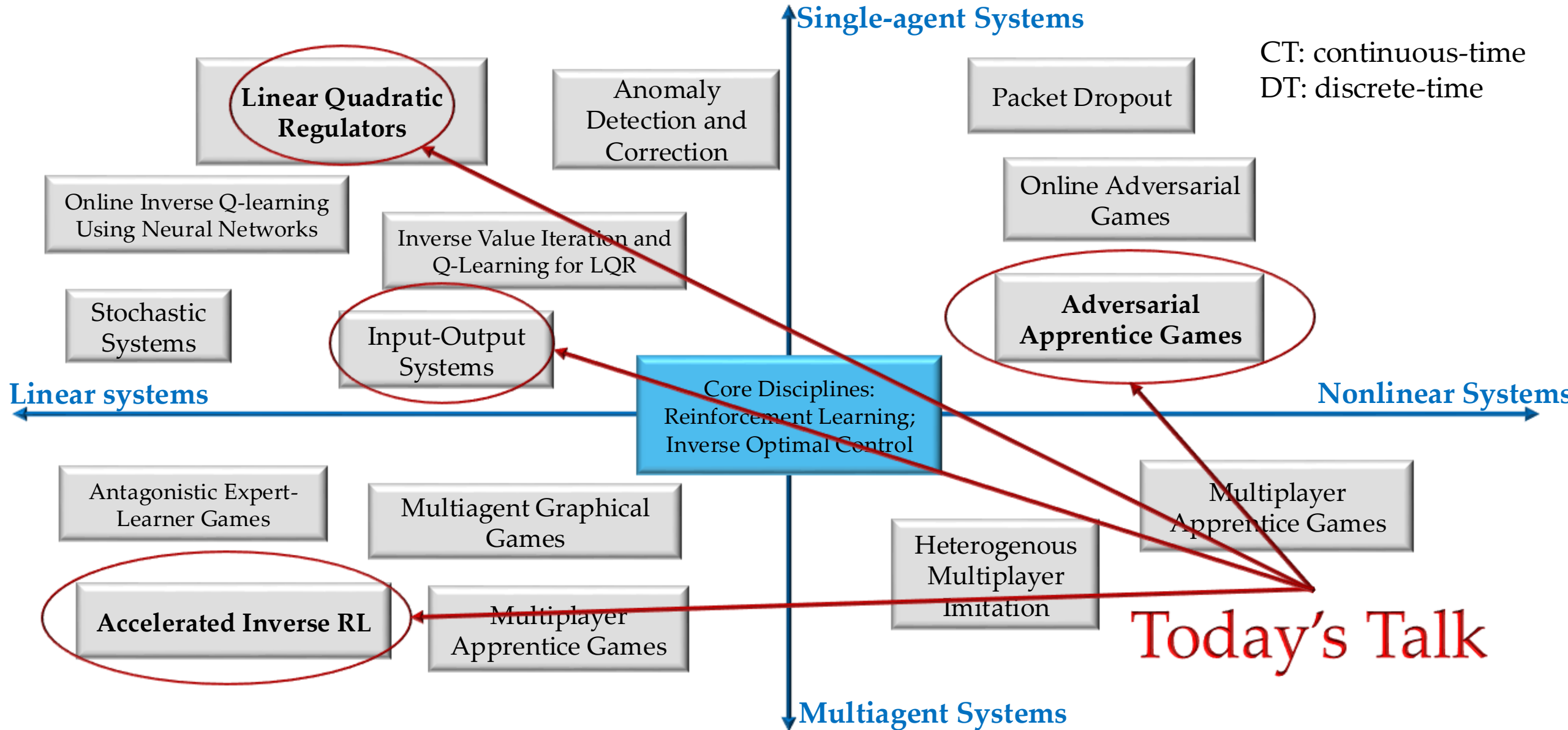
Applications to...



Inverse RL Diagram for Control Systems



Our Results of Inverse Reinforcement Learning for Control Systems

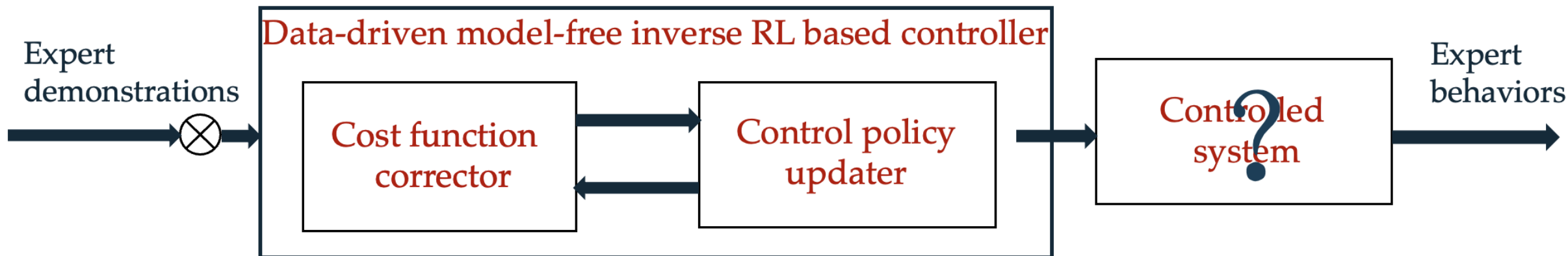


■ Controlled System (Learner)

Discrete-time linear system with unknown model / dynamics:

$$\begin{cases} x(k+1) = Ax(k) + Bu(k) \\ y(k) = Cx(k) \end{cases}$$

■ Control Problem



The expert demonstrations are exhibited by an Expert system with the same dynamics

$$\begin{cases} x_T(k+1) = Ax_T(k) + Bu_T(k) \\ y_T(k) = Cx_T(k) \end{cases}$$

where the control input u_T optimally corresponds to the **underlying expert cost function**

$$V_T(x_T(k), u_T(k)) = \sum_{j=k}^{\infty} (x_T(j)^T Q_T x_T(j) + u_T(j)^T R_T u_T(j))$$

which is given by

$$u_T = -K_T x_T = -(R_T + B^T P_T B)^{-1} B^T P_T A x_T$$

and satisfies the DT algebraic Riccati equation

$$Q_T - P_T + A^T P_T A - A^T P_T B (R_T + B^T P_T B)^{-1} B^T P_T A = 0.$$

or Lyapunov equation

$$Q_T + (A - BK_T)^T P_T (A - BK_T) + K_T^T P_T K_T - P_T = 0.$$

Only measurement data of expert's x_T, u_T are available

■ Problem

Using expert's data, find a group of $Q \geq 0$, $R > 0$ and P, K for learner to minimize the following index without knowing system dynamics A, B

$$J^* = \min_{Q, R} \sum_{t=0}^{\infty} \|x(k) - x_T(k)\| + \|u(k) - u_T(k)\|$$

$$\text{s.t. } x(k+1) = Ax(k) + Bu(k), \quad y(k) = Cx(k)$$

$$\text{Optimality condition} \quad \begin{cases} Q - P + A^T P A - A^T P B (R + B^T P B)^{-1} B^T P A = 0 \\ u(k) = -Kx(k) = -(R + B^T R B)^{-1} B^T P A x(k) - R^{-1} B^T P x \end{cases}$$

■ Controller Design

Goal solution condition

Theorem: If there exists a group of Q and R satisfying both (1) and the Lyapunov equation

$$Q + (A - BK_T)^T P (A - BK_T) + K_T^T R K_T - P = 0$$

then, the control gain K in (2) equals expert's K_T and minimizes J .

■ Model-based algorithm

1. **Initialization:** select $R > 0$, an initial $Q^0 = 0$, and a threshold $\varepsilon > 0$, set $i = 0$, $\alpha \in (0, 1]$

2. **Control policy evaluation and update:** update P^{i+1} by

$$Q^i - P^{i+1} + (A - BK_T)^T P^{i+1} (A - BK_T) + K_T^T R K_T = 0.$$

and control input u^{i+1} by

$$u^{i+1} = -K^{i+1}x = -(R + B^T P^{i+1} B)^{-1} B^T P^{i+1} A x$$

} **Modified RL policy iteration**

3. **Cost function weight correction:** update Q^{i+1} by

$$Q^{i+1} - (1 - \alpha)Q^i + \alpha(-P^{i+1} + A^T P^{i+1} A - A^T P^{i+1} B(R + B^T P^{i+1} B)^{-1} B^T P^{i+1} A) = 0 \quad \text{Inverse optimal control}$$

4. Set $i = i + 1$, repeat 1-3 until $\|Q^{i+1} - Q^i\| \leq \varepsilon$.

1

Evaluate Trail Value Function: P^{i+1}

$$Q^i - P^{i+1} + (A - BK_T)^T P^{i+1} (A - BK_T) + K_T^T R K_T = 0.$$

2

Improve Policy: K^{i+1}

$$u^{i+1} = -K^{i+1}x = -(R + B^T P^{i+1} B)^{-1} B^T P^{i+1} A x$$

3

Improve Cost Weights: Q^{i+1}

$$Q^{i+1} - (1 - \alpha)Q^i + \alpha(-P^{i+1} + A^T P^{i+1} A - A^T P^{i+1} B(R + B^T P^{i+1} B)^{-1} B^T P^{i+1} A) = 0$$

■ Model-free inverse reinforcement Q-learning algorithm

Kernel matrix evaluation and control policy update: solve H^{i+1} by

$$\begin{bmatrix} x_T(k) \\ u_T(k) \end{bmatrix}^T H^{i+1} \begin{bmatrix} x_T(k) \\ u_T(k) \end{bmatrix} - \begin{bmatrix} x_T(k+1) \\ u_T(k+1) \end{bmatrix}^T H^{i+1} \begin{bmatrix} x_T(k+1) \\ u_T(k+1) \end{bmatrix} = x_T(k)^T Q^i x_T(k) + u_T(k)^T R u_T(k).$$

Update control policy u^{i+1} by

$$u^{i+1} = -K^{i+1}x = -(H_{uu}^{i+1})^{-1} H_{ux}^{i+1}x$$

Cost function weight correction: solve Q^{i+1} by

$$x(k)^T Q^{i+1} x(k) = (1-\alpha)x(k)^T Q^i x(k) - \alpha u(k)^T R u(k) + \alpha \begin{bmatrix} x(k-1) \\ u(k-1) \end{bmatrix}^T H_p^{i+1} \begin{bmatrix} x(k-1) \\ u(k-1) \end{bmatrix} - \alpha \begin{bmatrix} x(k) \\ u(k) \end{bmatrix}^T H_p^{i+1} \begin{bmatrix} x(k) \\ u(k) \end{bmatrix}$$

■ Simulation Experiment

The dynamics of the Expert and Learner systems are (F-16 aircraft system dynamics)

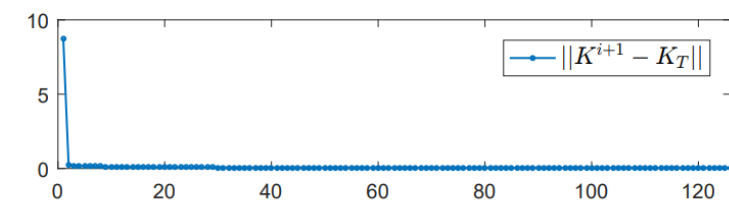
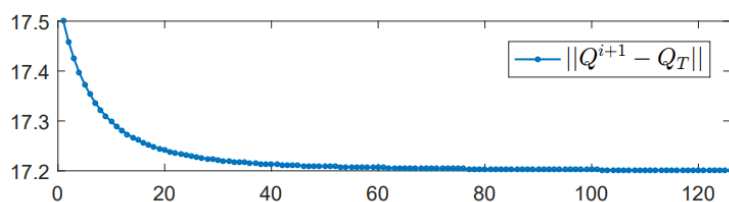
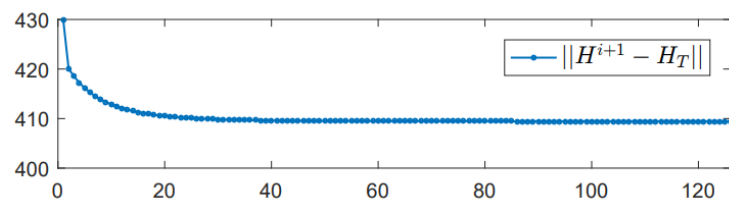
$$A = \begin{bmatrix} 0.906488 & 0.0816012 & -0.0005 \\ 0.0741349 & 0.90121 & -0.000708383 \\ 0 & 0 & 0.132655 \end{bmatrix}, B = \begin{bmatrix} -0.00150808 \\ -0.0096 \\ 0.867345 \end{bmatrix}, C = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0.6 & 0 \\ -1.26 & -0.9788 & 0.4852 \end{bmatrix}.$$

$$\text{Expert's input: } u_T(k) = -K_T x_T(k) = -\begin{bmatrix} -0.1688 & -0.2009 & 0.1285 \end{bmatrix} x_T(k) \quad \text{Expert's weights: } Q_T = \begin{bmatrix} 15 & 0 & 0 \\ 0 & 18 & 0 \\ 0 & 0 & 14 \end{bmatrix}, R_T = 2I_1$$

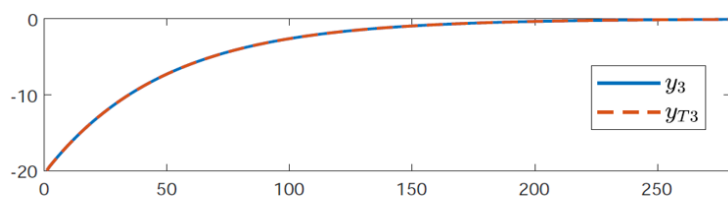
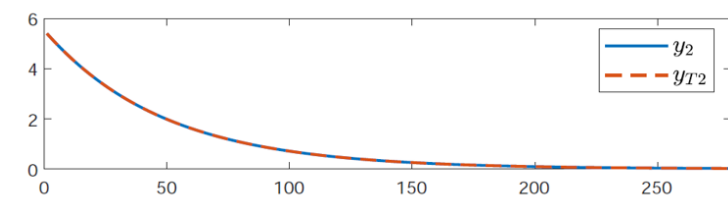
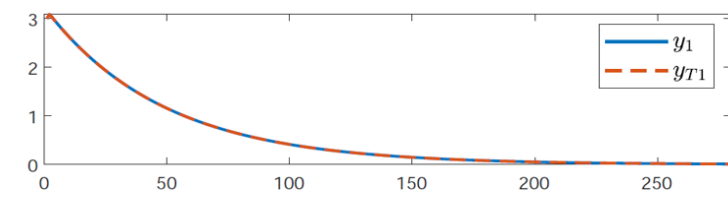
$$\text{Initial parameters for learning: } Q^0 = \begin{bmatrix} 0.1 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}, R = I_1, \alpha=1 \quad \text{Threshold: } \varepsilon = 6 \times 10^{-4}$$

Simulation goal: Using expert's demonstrations, find controlled system a group of Q, R, K using the data-driven model-free IRL algorithm to realize trajectory imitation.

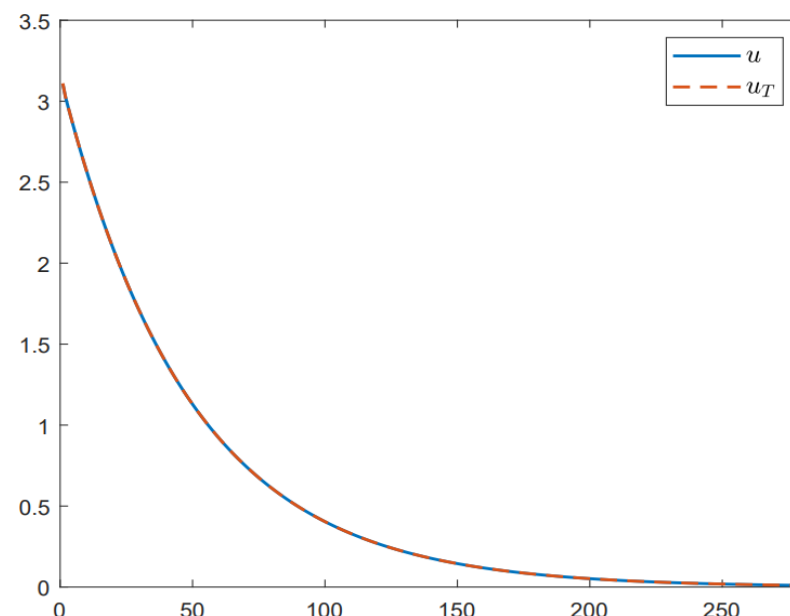
■ Simulation Experiment



Iteration Steps



Time



Time

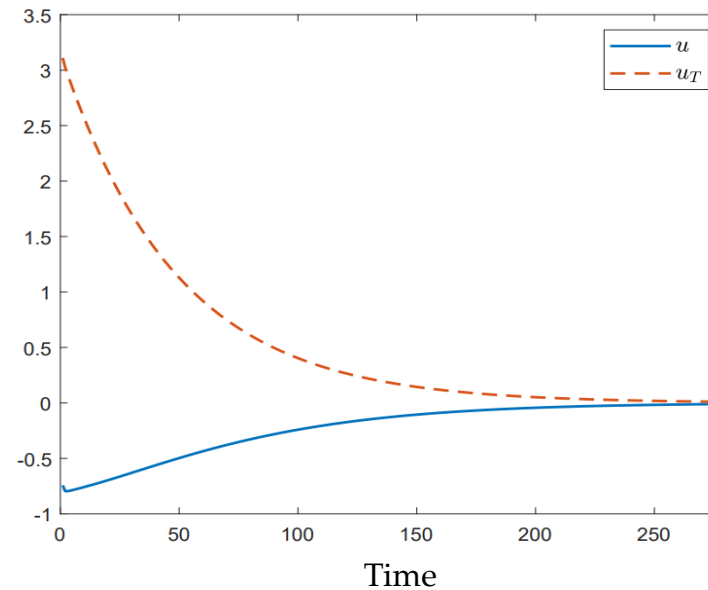
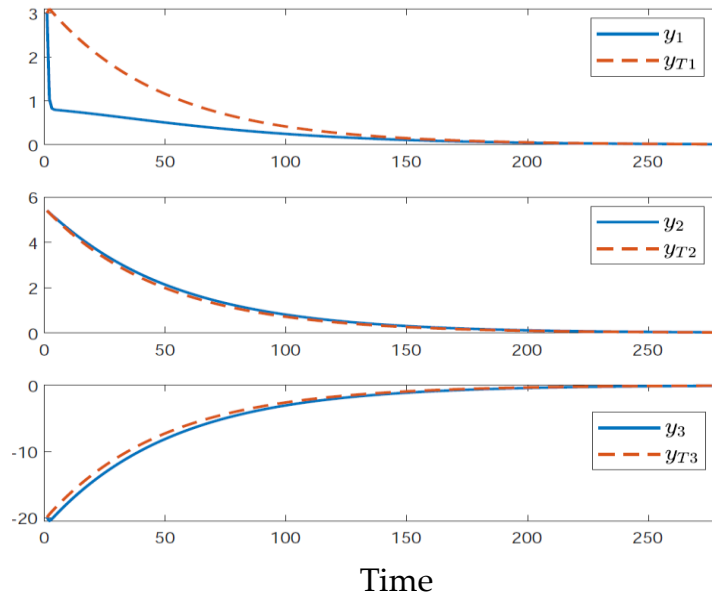
■ Simulation Experiment (for control performance improvement by correcting cost function)

Comparison method 1:

B. Kiumarsi, F. L. Lewis, H. Modares, A. Karimpour, and M. B. Naghibi-Sistani, "Reinforcement Q-learning for optimal tracking control of linear discrete-time systems with unknown dynamics," Automatica, vol. 50, no. 4, pp. 1167–1175, 2014.

Parameter setting :

$$Q = \begin{bmatrix} 0.1 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{bmatrix}, R = I_1, \gamma = 0.9$$

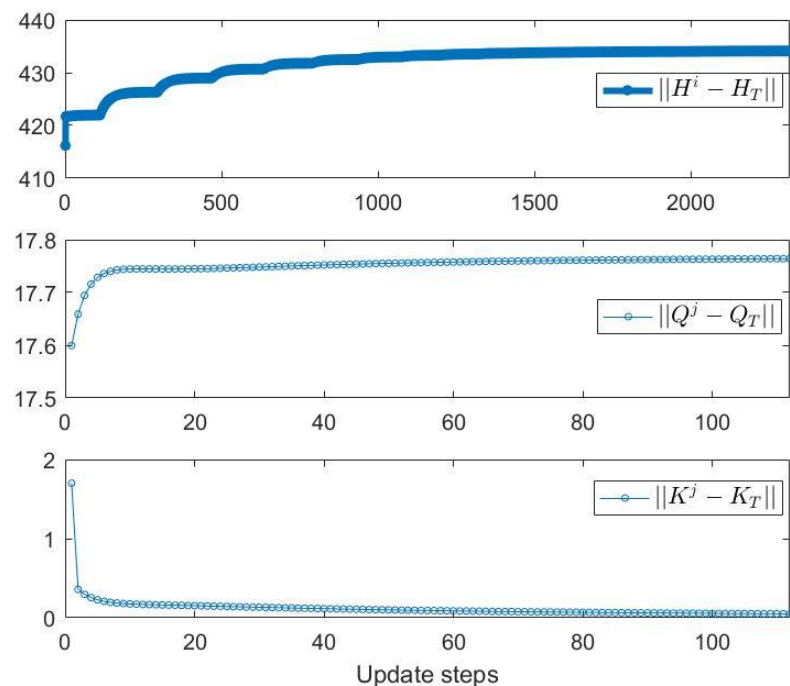


	Output RMSE	Input RMSE
Inverse	5.0476×10^{-5}	1.1473×10^{-4}
RL		
Com 1	0.4053	1.2422

■ Simulation Experiment (for learning efficiency improvement by the single-loop learning structure)

Comparison method 2:

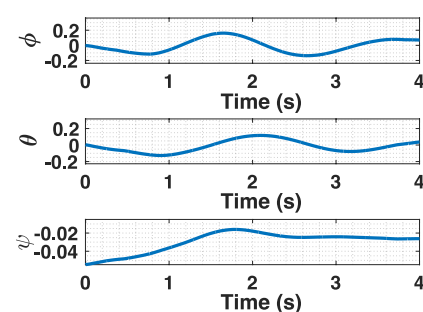
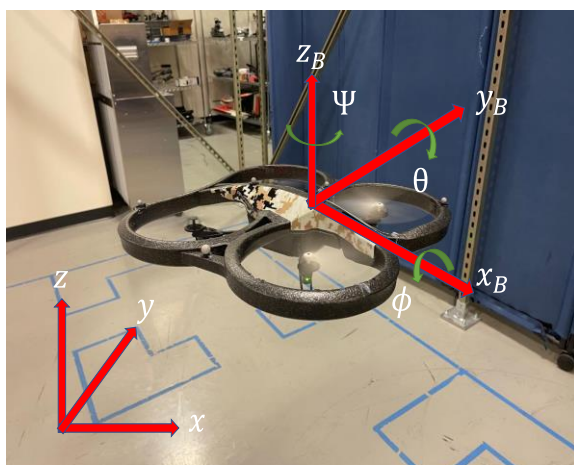
Apply the bilevel two-loop learning structure [1, 2] to the proposed formulas.



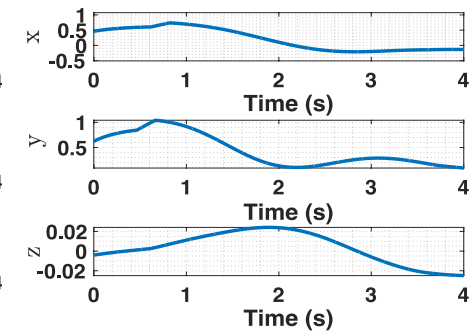
	Outer iterations j	Inner iterations i	Total	time
Inverse RL	\	\	81	10.360s
Com 2	112	2315	2427	190.576s

[1] P. Abbeel and A. Y. Ng, "Apprenticeship Learning via Inverse Reinforcement Learning," the 21st International Conference on Machine Learning, Banff, Canada, 2004.

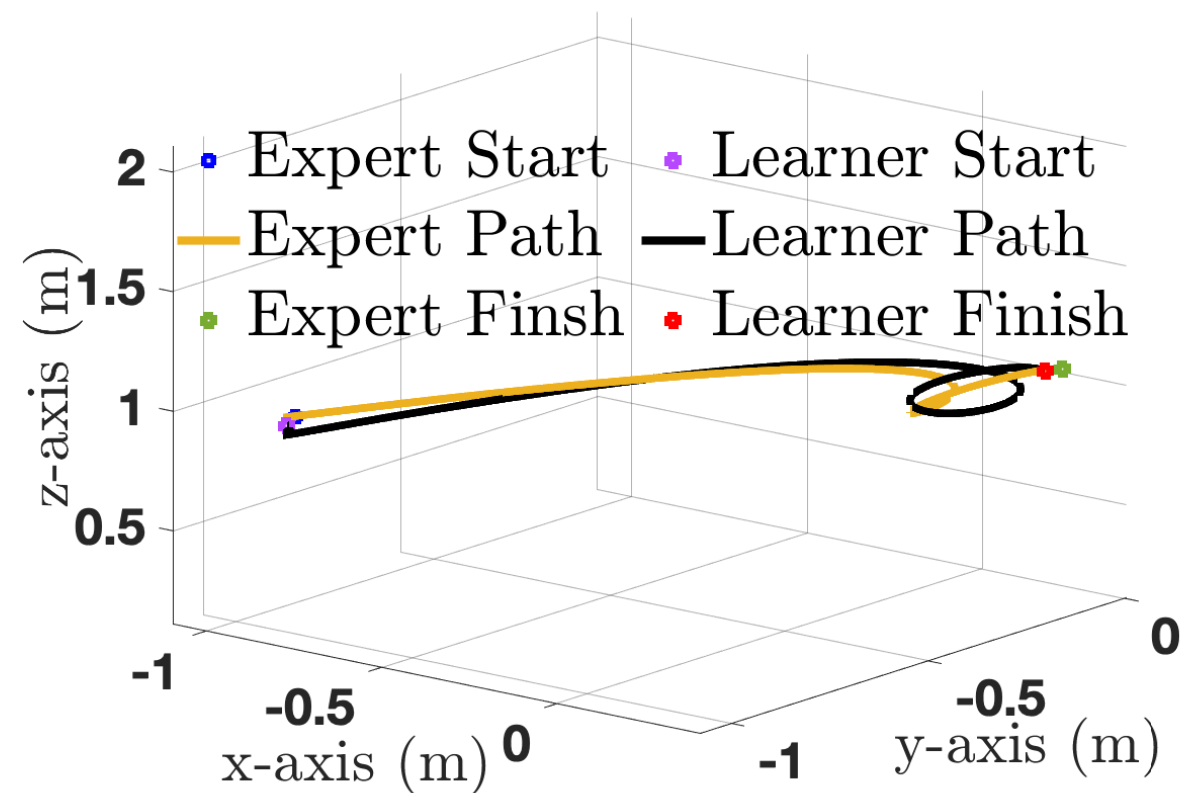
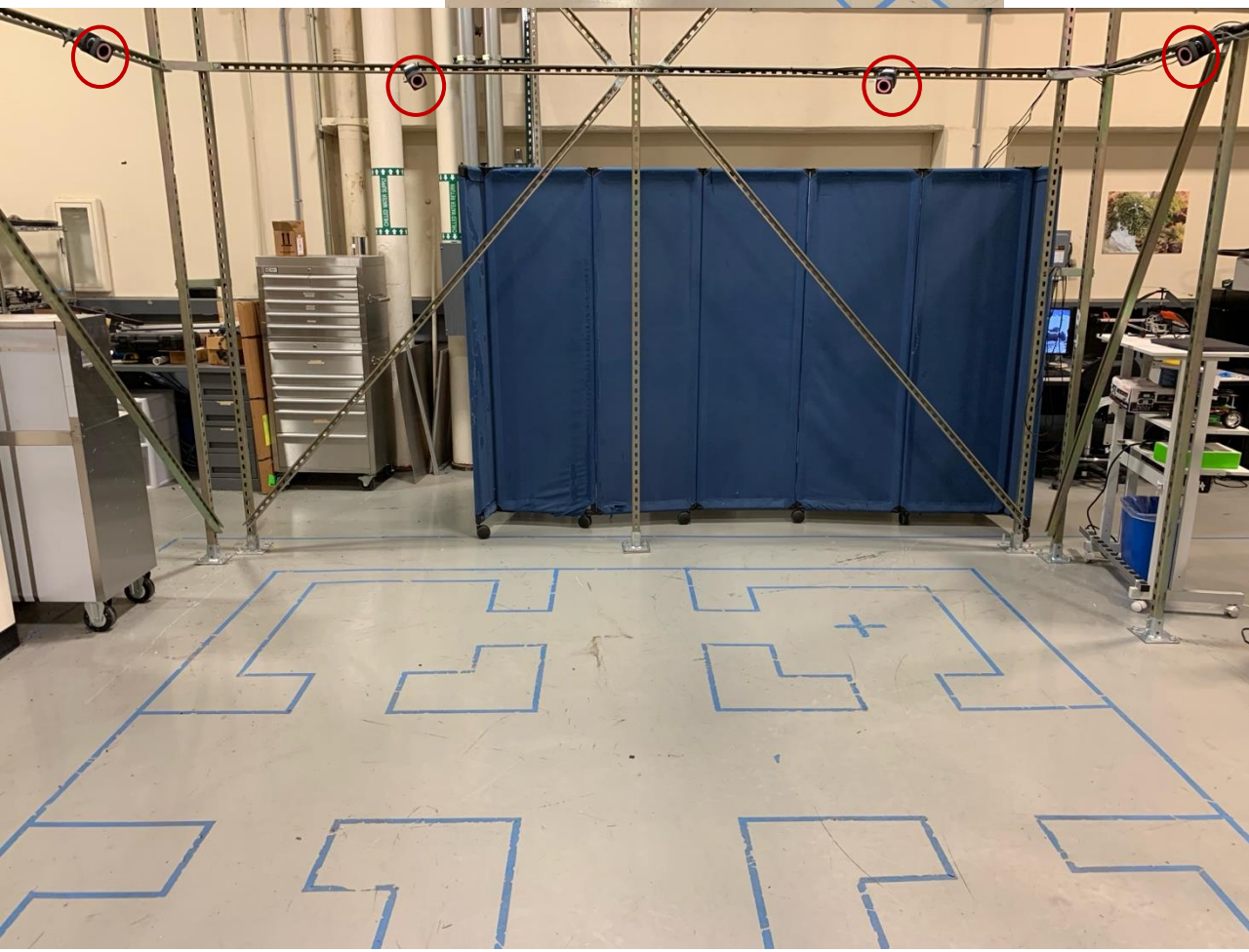
[2] K. Mombaur, A. Truong, J P Laumond. From Human to Humanoid Locomotion—An Inverse Optimal Control Approach. Auton Robots, 2010, 28:369–383.



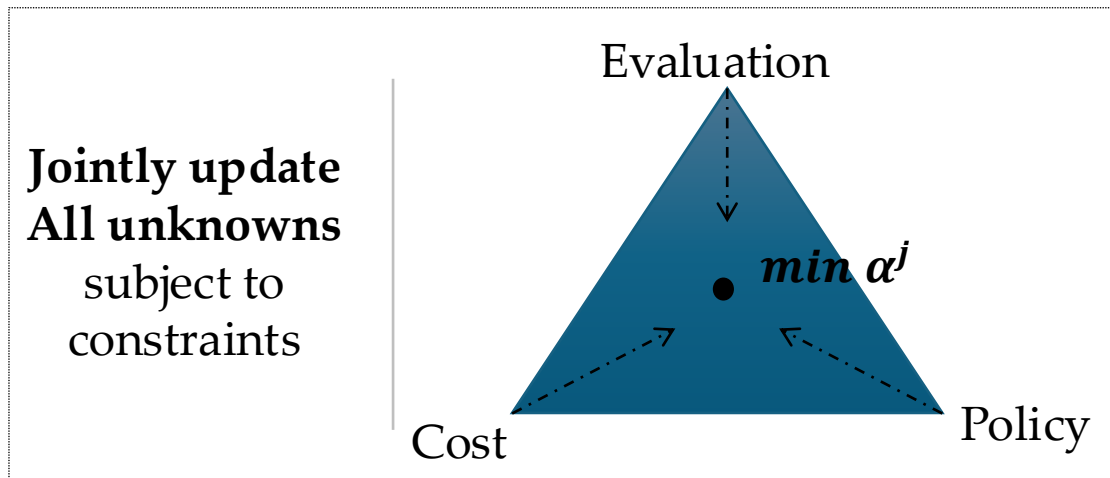
Euler angles



Cartesian Position



Convex Optimization based Inverse RL



α^*



linear model
best-fit cost weights
optimal policy

Iterative Data-Driven Convex Optimization

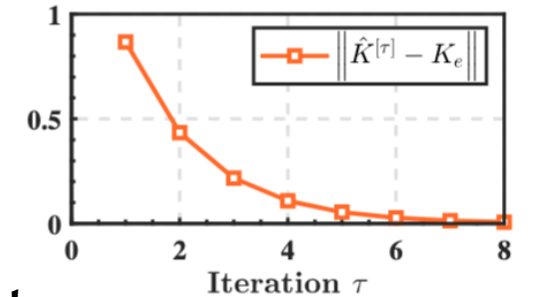
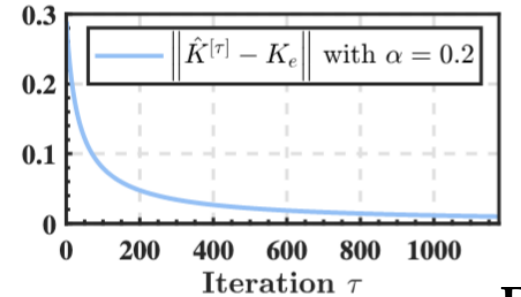
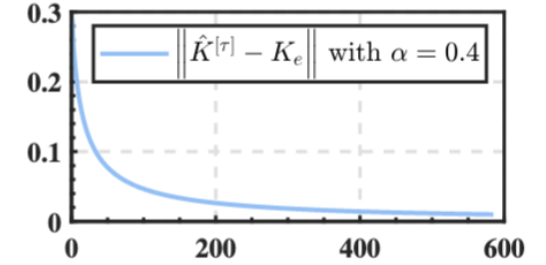
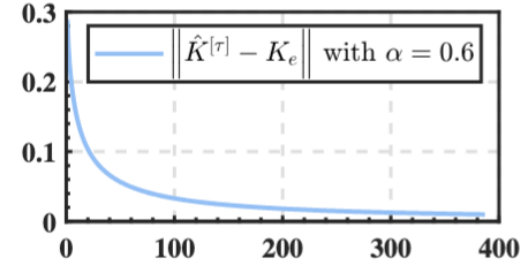
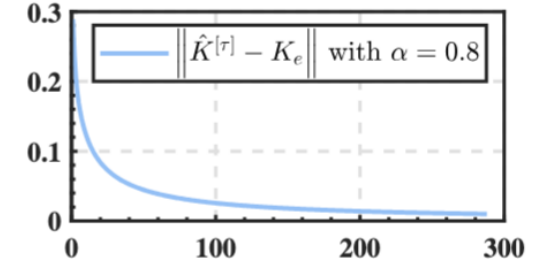
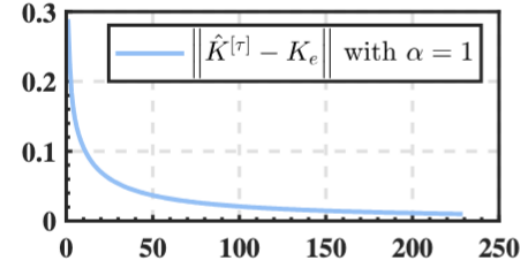
Allow for constraints
Find best-fit cost without requiring the optimal expert

Convex Optimization

$$\begin{aligned}
 \text{Step 1 } (M^{j+1}, \tilde{Q}_e^j) &= \arg \min \text{tr}((\alpha^j)^2) \\
 \text{s.t. } \left\{ \begin{aligned}
 Z_k^\top M^{j+1} Z_k &= z_k^\top \tilde{Q}_e^j z_k + u_k^\top R u_k + Z_{k+1}^\top M^{j+1} Z_{k+1} \\
 Z_k^\top S^{j+1} Z_k &= z_k^\top \tilde{Q}_e^j z_k + u_k^\top R u_k + (Z_{k+1}^{j+1})^\top S^{j+1} Z_{k+1}^{j+1} \\
 S^{j+1} &\succeq M^{j+1} \succ \mathbf{0} \\
 \alpha^j I &\succeq F(Q^j, R^j) \succeq I
 \end{aligned} \right. \\
 \text{Step 2 } K_r^{j+1} &= -(M_{uu}^{j+1})^{-1} M_{uz}^{j+1}
 \end{aligned}$$

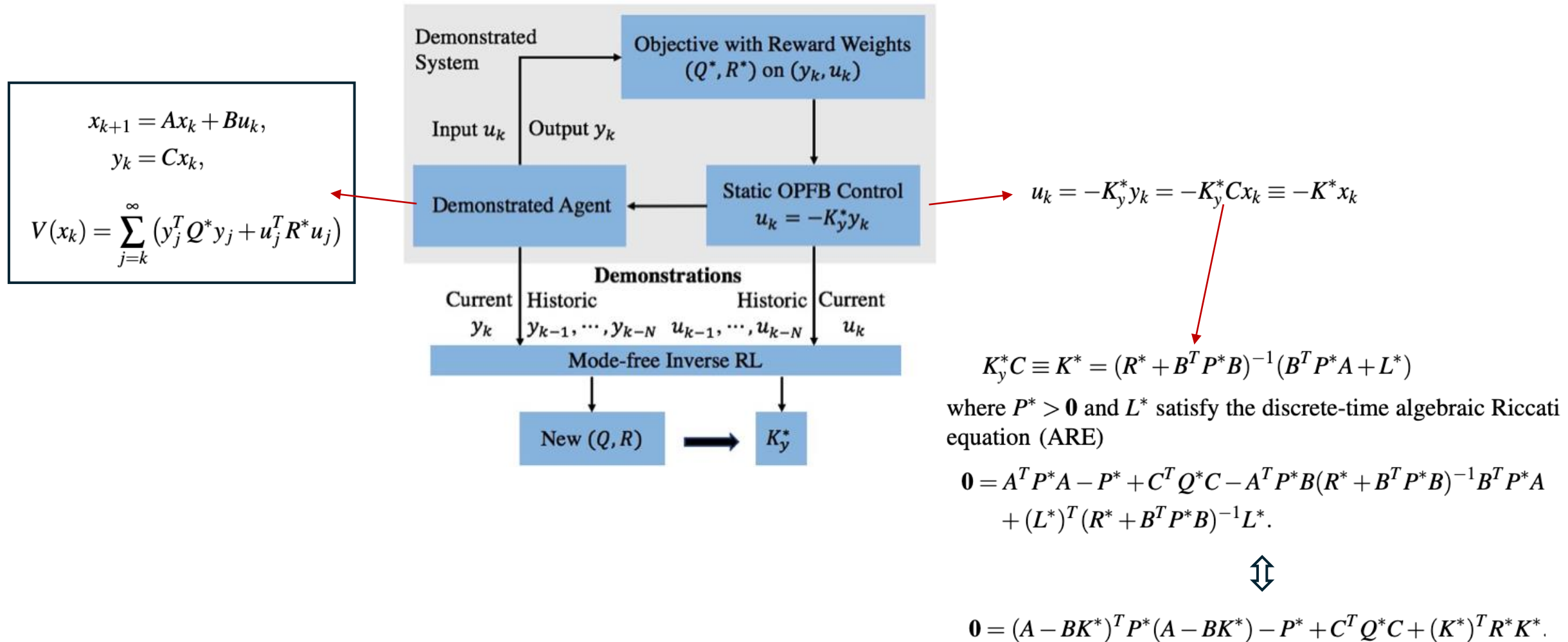
Adaptive Dynamic Programming
Inverse Optimal Control

Model free and update Q and R



Faster

Model-Free Inverse RL for Output-Feedback Control Systems



■ Algorithm Design

Goal: Using expert's demonstrations, find a group of Q, R_e such that

$$\begin{aligned} \mathbf{0} &= (A - BK^*)^T P_e (A - BK^*) - P_e + C^T Q_e C + (K^*)^T R_e K^*, \\ K_e C &= (R_e + B^T P_e B)^{-1} (B^T P_e A + L_e), \\ L_e &= (R_e + B^T P_e B) K_e C - B^T P_e A. \end{aligned}$$

Algorithm 1 Model-Based Inverse RL Algorithm for Discrete-Time Static OPFB Control Systems

1: **Initialization:** Select arbitrary $R_e = R_e^T \in \mathbb{R}^{m \times m} > \mathbf{0}$, a small threshold δ , and a tuning parameter $\alpha \in (0, 1]$. Give (A, B, C) and K^* . Set $i = 0$, $Q_0 = \mathbf{0}$, and $L_0 = \mathbf{0}$.

2: **Evaluate P_i using Lyapunov equation**

$$\mathbf{0} = (A - BK^*)^T P_i (A - BK^*) - P_i + C^T Q_i C + (K^*)^T R_e K^*.$$

3: **Improve K_{i+1} and L_{i+1} using static OPFB control**

$$\begin{aligned} K_{i+1} &= (R_e + B^T P_i B)^{-1} (B^T P_i A + L_i) C^T (C C^T)^{-1}, \\ L_{i+1} &= (R_e + B^T P_i B) K_{i+1} C - B^T P_i A. \end{aligned}$$

4: **Improve Q_{i+1} relying on inverse optimal control**

$$\begin{aligned} Q_{i+1} &= \alpha (C C^T)^{-1} C [P_i - (A - BK_{i+1} C)^T P_i (A - BK_{i+1} C) \\ &\quad - C^T K_{i+1}^T R_e K_{i+1} C - L_i^T K^* - (K^*)^T L_i + L_i^T K_{i+1} C \\ &\quad + C^T K_{i+1}^T L_i] C^T (C C^T)^{-1} + (1 - \alpha) Q_i. \end{aligned}$$

5: **Stop if $\|K_{i+1} - K^*\| \leq \delta$, or set $i \rightarrow i + 1$ and go to Step 2.**

Only measurement data of expert's u_k, y_k are available

■ Model-free algorithm framework

Replace

2: Evaluate P_i using Lyapunov equation

$$\mathbf{0} = (A - BK^*)^T P_i (A - BK^*) - P_i + C^T Q_i C + (K^*)^T R_e K^*.$$

Model-free version to update P^i ($\bar{H}^i$):

System State
Reconstruction:

$$x_k = D_u U_{k-1:k-N} + D_y Y_{k-1:k-N}$$

$$U_{k-1:k-N} = \begin{bmatrix} u_{k-1} \\ u_{k-2} \\ \vdots \\ u_{k-N} \end{bmatrix}, \quad Y_{k-1:k-N} = \begin{bmatrix} y_{k-1} \\ y_{k-2} \\ \vdots \\ y_{k-N} \end{bmatrix}$$

$$D_u = V_N - A^N (M_N^T M_N)^{-1} M_N^T T_N, \\ D_y = A^N (M_N^T M_N)^{-1} M_N^T,$$

$$V_N = \begin{bmatrix} B & AB & \dots & A^{N-1}B \end{bmatrix}, \\ M_N = \begin{bmatrix} CA^{N-1} \\ CA^{N-2} \\ \vdots \\ CA \\ C \end{bmatrix}, \quad T_N = \begin{bmatrix} \mathbf{0} & CB & CAB & \dots & CA^{N-2}B \\ \mathbf{0} & \mathbf{0} & CB & \dots & CA^{N-3}B \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ \mathbf{0} & \dots & \dots & \mathbf{0} & CB \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix}$$

$$\begin{bmatrix} x_k \\ u_k \end{bmatrix}^T H^i \begin{bmatrix} x_k \\ u_k \end{bmatrix} = \begin{bmatrix} x_{k+1} \\ u_{k+1} \end{bmatrix}^T H^i \begin{bmatrix} x_{k+1} \\ u_{k+1} \end{bmatrix} + x_k^T C^T Q_i C x_k + u_k^T R_e u_k$$

$$H^i \equiv \begin{bmatrix} H_{xx}^i & H_{xu}^i \\ H_{ux}^i & H_{uu}^i \end{bmatrix} \in \mathbb{R}^{(n+m) \times (n+m)},$$

$$H_{xx}^i \equiv C^T Q_i C + A^T P_i A \in \mathbb{R}^{n \times n},$$

$$H_{xu}^i = (H_{ux}^i)^T \equiv A^T P_i B \in \mathbb{R}^{n \times m},$$

$$H_{uu}^i \equiv R_e + B^T P_i B \in \mathbb{R}^{m \times m}.$$

$$z_k = \begin{bmatrix} U_{k-1:k-N}^T & Y_{k-1:k-N}^T & u_k^T \end{bmatrix}^T$$



$$z_k^T \bar{H}^i z_k - z_{k+1}^T \bar{H}^i z_{k+1} = y_k^T Q_i y_k + u_k^T R_e u_k.$$

$$\bar{H}^i \equiv \begin{bmatrix} H_{UU}^i & H_{UY}^i & H_{Uu}^i \\ H_{YU}^i & H_{YY}^i & H_{Yu}^i \\ H_{uU}^i & H_{uY}^i & H_{uu}^i \end{bmatrix} \in \mathbb{R}^{(mN+pN+m) \times (mN+pN+m)}, \\ H_{UU}^i \equiv D_u^T H_{xx}^i D_u \in \mathbb{R}^{mN \times mN}, \quad H_{UY}^i \equiv D_u^T H_{xx}^i D_y \in \mathbb{R}^{mN \times pN}, \\ H_{Uu}^i \equiv D_u^T H_{xu}^i \in \mathbb{R}^{mN \times m}, \quad H_{YY}^i \equiv D_y^T H_{xx}^i D_y \in \mathbb{R}^{pN \times pN}, \\ H_{Yu}^i \equiv D_y^T H_{xu}^i \in \mathbb{R}^{pN \times m}, \quad H_{YU}^i = (H_{UY}^i)^T \in \mathbb{R}^{pN \times mN}, \\ H_{uU}^i = (H_{Uu}^i)^T \in \mathbb{R}^{m \times mN}, \quad H_{uY}^i = (H_{Yu}^i)^T \in \mathbb{R}^{m \times pN}.$$

Algorithm 1 Model-Based Inverse RL Algorithm for Discrete-Time Static OPFB Control Systems

1: **Initialization:** Select arbitrary $R_e = R_e^T \in \mathbb{R}^{m \times m} > \mathbf{0}$, a small threshold δ , and a tuning parameter $\alpha \in (0, 1]$. Give (A, B, C) and K^* . Set $i = 0$, $Q_0 = \mathbf{0}$, and $L_0 = \mathbf{0}$.

2: **Evaluate P_i using Lyapunov equation**

$$\mathbf{0} = (A - BK^*)^T P_i (A - BK^*) - P_i + C^T Q_i C + (K^*)^T R_e K^*.$$

3: **Improve K_{i+1} and L_{i+1} using static OPFB control**

$$K_{i+1} = (R_e + B^T P_i B)^{-1} (B^T P_i A + L_i) C^T (C C^T)^{-1},$$

$$L_{i+1} = (R_e + B^T P_i B) K_{i+1} C - B^T P_i A.$$

4: **Improve Q_{i+1} relying on inverse optimal control**

$$Q_{i+1} = \alpha (C C^T)^{-1} C [P_i - (A - BK_{i+1} C)^T P_i (A - BK_{i+1} C) - C^T K_{i+1}^T R_e K_{i+1} C - L_i^T K^* - (K^*)^T L_i + L_i^T K_{i+1} C + C^T K_{i+1}^T L_i] C^T (C C^T)^{-1} + (1 - \alpha) Q_i.$$

5: **Stop if $\|K_{i+1} - K_y^*\| \leq \delta$, or set $i \rightarrow i + 1$ and go to Step 2.**

Model-free version to update K_{i+1} and L_{i+1} :

$$K_{i+1} C x_k = K_{i+1} y_k$$

$$= (R_e + B^T P_i B)^{-1} (B^T P_i A + L_i) x_k$$

$$= (R_e + B^T P_i B)^{-1} (B^T P_i A D_u U_{k-1:k-N} + B^T P_i A D_y Y_{k-1:k-N}) + (R_e + B^T P_i B)^{-1} L_i x_k$$

$$= (H_{uu}^i)^{-1} (H_{uU}^i U_{k-1:k-N} + H_{uY}^i Y_{k-1:k-N}) + (H_{uu}^i)^{-1} L_i(x_k),$$

$$L_{i+1}(x_k) = [(R_e + B^T P_i B) K_{i+1} C - B^T P_i A] x_k$$

$$= (R_e + B^T P_i B) K_{i+1} C x_k - (B^T P_i A D_u U_{k-1:k-N} + B^T P_i A D_y Y_{k-1:k-N})$$

$$= H_{uu}^i K_{i+1} y_k - (H_{uU}^i U_{k-1:k-N} + H_{uY}^i Y_{k-1:k-N}).$$

Algorithm 1 Model-Based Inverse RL Algorithm for Discrete-Time Static OPFB Control Systems

1: **Initialization:** Select arbitrary $R_e = R_e^T \in \mathbb{R}^{m \times m} > \mathbf{0}$, a small threshold δ , and a tuning parameter $\alpha \in (0, 1]$. Give (A, B, C) and K^* . Set $i = 0$, $Q_0 = \mathbf{0}$, and $L_0 = \mathbf{0}$.

2: **Evaluate P_i using Lyapunov equation**

$$\mathbf{0} = (A - BK^*)^T P_i (A - BK^*) - P_i + C^T Q_i C + (K^*)^T R_e K^*.$$

3: **Improve K_{i+1} and L_{i+1} using static OPFB control**

$$K_{i+1} = (R_e + B^T P_i B)^{-1} (B^T P_i A + L_i) C^T (C C^T)^{-1},$$

$$L_{i+1} = (R_e + B^T P_i B) K_{i+1} C - B^T P_i A.$$

4: **Improve Q_{i+1} relying on inverse optimal control**

$$\begin{aligned} Q_{i+1} = & \alpha (C C^T)^{-1} C [P_i - (A - BK_{i+1} C)^T P_i (A - BK_{i+1} C) \\ & - C^T K_{i+1}^T R_e K_{i+1} C - L_i^T K^* - (K^*)^T L_i + L_i^T K_{i+1} C \\ & + C^T K_{i+1}^T L_i] C^T (C C^T)^{-1} + (1 - \alpha) Q_i. \end{aligned}$$

5: **Stop if $\|K_{i+1} - K_y^*\| \leq \delta$, or set $i \rightarrow i + 1$ and go to Step 2.**

Model-free version to update Q_{i+1} :

$$C^T Q_{i+1} C = \alpha (K_{i+1} C - K^*)^T (R_e + B^T P_i B) (K_{i+1} C - K^*) + C^T Q_i C.$$

Multiply x_k

$$\begin{aligned} y_k^T Q_{i+1} y_k &= \alpha (K_{i+1} C x_k + u_k)^T H_{uu}^i (K_{i+1} C x_k + u_k) + y_k^T Q_i y_k \\ &= \alpha (K_{i+1} y_k + u_k)^T H_{uu}^i (K_{i+1} y_k + u_k) + y_k^T Q_i y_k. \end{aligned}$$

Model-Free Inverse RL for Continuous-time Linear Systems

For CT linear system with unknown dynamics:

$$\dot{x}(x) = Ax(t) + Bu(t)$$

Algorithm 1 Model-based Inverse Value Iteration Algorithm

Input: Any $R = R^T > 0$. Threshold $\varepsilon > 0$. (A, B) . K_d .

Output: Converged values Q^∞ and K^∞ .

1: **Initialization:** $i = 0$. $Q^0 \geq 0$.

2: **Evaluate P^i using Lyapunov Equation**

$$0 = (A - BK_d)^T P^i + P^i (A - BK_d) + Q^i + (K_d)^T R K_d.$$

3: **Improve K^{i+1} using Optimal Control**

$$K^{i+1} = R^{-1} B^T P^i.$$

4: **Improve Q^{i+1} using IOC**

$$Q^{i+1} = -(A - BK^{i+1})^T P^i - P^i (A - BK^{i+1}) - (K^{i+1})^T R K^{i+1}.$$

5: **Stop if $\|Q^{i+1} - Q\| \leq \varepsilon$, or set $i \rightarrow i + 1$ and go to Step 2.**

Algorithm 2 Model-free Inverse Q -learning Algorithm

Input: Any $R = R^T > 0$. Threshold $\varepsilon > 0$, $z = [x_d^T, u_d^T]^T$

Output: Converged values Q^∞ and K^∞ .

1: **Initialization:** $i = 0$. $Q^0 \geq 0$.

2: **Evaluate H^i using Integral RL by**

$$\begin{aligned} & z^T(t) H^i z(t) - z^T(t + \Delta t) H^i z(t + \Delta t) \\ &= \int_t^{t+\Delta t} (x_d^T Q^i x_d + u_d^T R u_d) d\tau. \end{aligned}$$

3: **Evaluate u^{i+1} (i.e., K^{i+1}) using Optimal Control by**

$$u^{i+1} = -(H_{u_d u_d}^i)^{-1} (H_{x_d u_d}^i)^T x_d.$$

4: **Improve Q^{i+1} using IOC by**

$$\begin{aligned} & \int_t^{t+\Delta t} x_d^T Q^{i+1} x_d d\tau \\ &= \int_t^{t+\Delta t} x_d^T Q^i x_d d\tau + \int_t^{t+\Delta t} (u_d - u^{i+1})^T R (u_d - u^{i+1}) d\tau. \end{aligned}$$

5: **Stop if $\|Q^{i+1} - Q^i\| \leq \varepsilon$, or set $i \rightarrow i + 1$ and go to Step 2.**

Inverse RL for Systems with Disturbances

For CT linear system with disturbances:

$$\dot{x}(x) = Ax(t) + Bu(t) + Dd(t)$$

Algorithm 1 Model-based IRL for inverse H_∞ control

- 1) Initialize with $R > 0$, $\gamma > 0$, $Q^0 > 0$, $L^0 = 0$, and $i = 0$.
- 2) **Policy correction:** Update policy P^i by

$$\begin{aligned} (A - BK_T)^T P^i + P^i (A - BK_T) \\ = -Q^i - K_T^T R K_T - \gamma^2 (L^i)^T L^i. \end{aligned}$$

- 3) **Action update:** Update u^{i+1} , K^{i+1} and L^{i+1} by

$$\begin{aligned} u^{i+1} &= -K^{i+1}x = -R^{-1}B^T P^i x, \\ L^{i+1} &= \frac{1}{\gamma^2} D^T P^i. \end{aligned}$$

- 4) **Cost function weight construction:** Update Q^{i+1} by

$$Q^{i+1} = -A^T P^i - P^i A + (K^{i+1})^T R K^{i+1} - \gamma^2 (L^{i+1})^T L^{i+1}.$$

- 5) Stop if it converges. Otherwise, set $i = i + 1$ and repeat steps 2 to 4.
-

Algorithm 2 Model-free IRL algorithm for inverse H_∞ control

- 1: Initialize with $R > 0$, $\gamma > 0$, $Q^0 > 0$, and $L^0 = 0$, and set $i = 0$.
- 2: **Policy correction:** Update policy P^i , control gain K^{i+1} and disturbance gain L^{i+1} by

$$\begin{aligned} & x_T(t+T)^T P^i x_T(t+T) - x_T(t)^T P^i x_T(t) \\ & - 2 \int_t^{t+T} e^T R K^{i+1} x_T d\tau - 2\gamma^2 \int_t^{t+T} d_T^T L^{i+1} x_T d\tau \\ & = - \int_t^{t+T} (x_T^T Q^i x_T + (u_T - e)^T R (u_T - e) \\ & \quad + \gamma^2 x_T^T (L^i)^T L^i x_T) d\tau. \end{aligned}$$

- 3: **Cost function weight construction:** Update cost function weight Q^{i+1} by

$$\begin{aligned} & \int_t^{t+T} x^T Q^{i+1} x d\tau \\ & = -[x(t+T)^T P^i x(t+T) - x(t)^T P^i x(t) \\ & \quad - \int_t^{t+T} (2u^T R K^{i+1} x + x^T (K^{i+1})^T R K^{i+1} x) d\tau \\ & \quad - \gamma^2 \int_t^{t+T} (2d^T L^{i+1} x - x^T (L^{i+1})^T L^{i+1} x) d\tau]. \end{aligned}$$

- 4: Stop if it converges. Otherwise, set $i = i + 1$ and repeat steps 2 to 3.
-

Data-Driven Inverse RL for Adversarial Apprentice Games

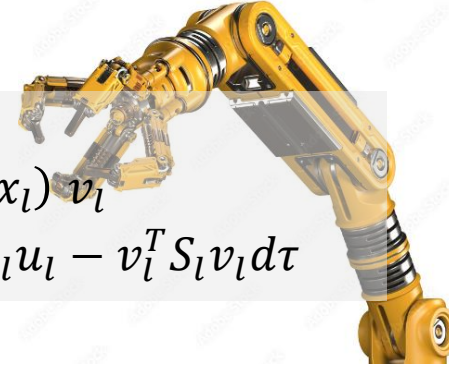


Expert

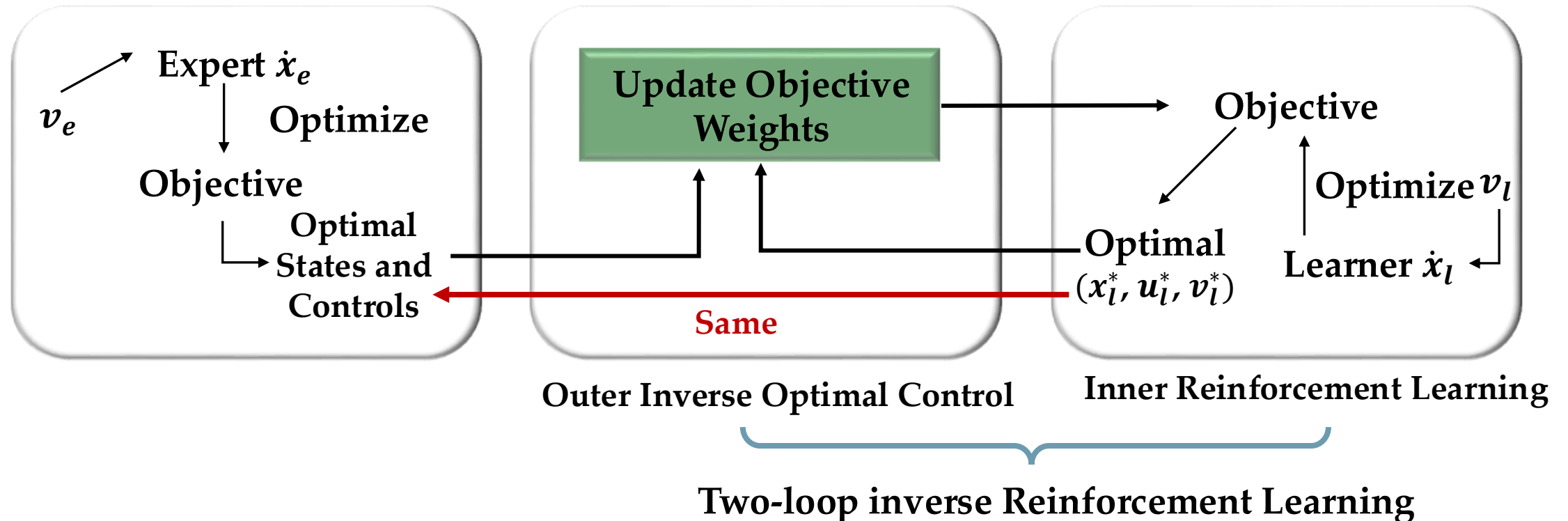
Dynamics: $\dot{x}_e = f(x_e) + g(x_e) u_e + h(x_e) v_e$
Objective: $V_e = \int_0^\infty x_e^T Q_e x_e + u_e^T R_e u_e - v_e^T S_e v_e d\tau$

Learner

Dynamics: $\dot{x}_l = f(x_l) + g(x_l) u_l + h(x_l) v_l$
Trail Objective: $V_l = \int_0^\infty x_l^T Q_l x_l + u_l^T R_l u_l - v_l^T S_l v_l d\tau$



UNKNOWN



Reinforcement Learning
(Off-Policy)

Inverse Optimal
Control

Algorithm 1 Model-free Integral Inverse RL Algorithm for Adversarial Apprentice Game

1: **Initialization:** select $Q_l^0 \geq 0$, $R_l > 0$, $S_l > 0$, initial stabilizing u_l^{00} , and small thresholds ε_1 , ε_2 . Set $j = 0$. Apply stabilizing u_l to the learner dynamics (27).

2: **Outer j iteration loop based on inverse optimal control**

3: **Inner i iteration loop using optimal control:** given j , set $i = 0$, and use initial stabilizing u_l^{j0} .

4: Off-policy Integral RL for solving V_l^j , u_l^j and v_l^j

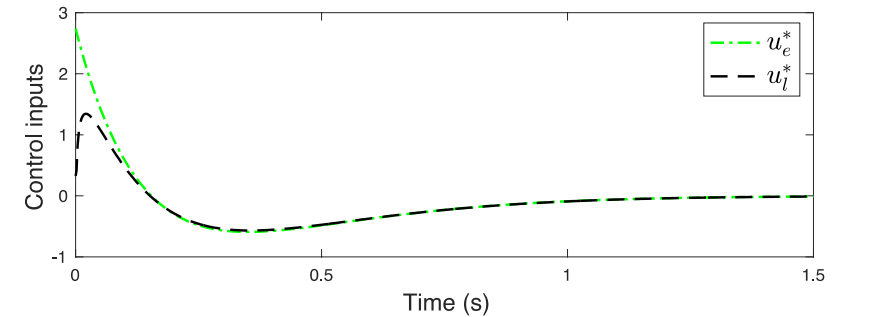
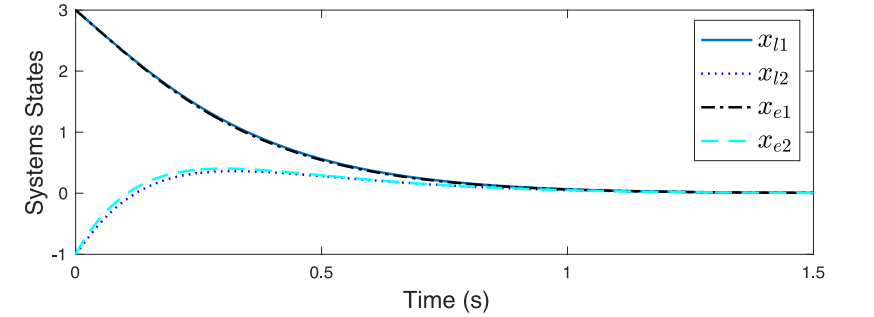
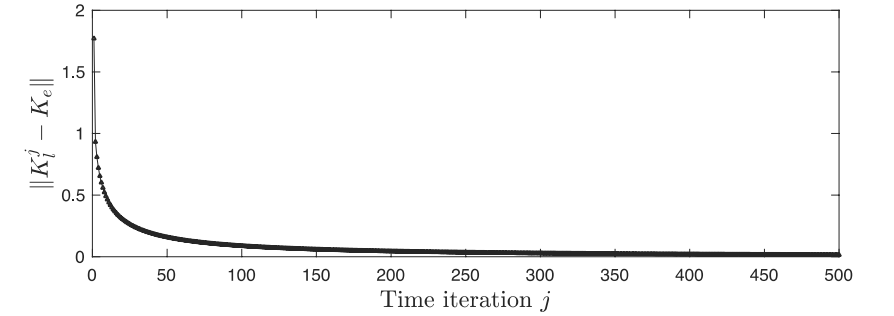
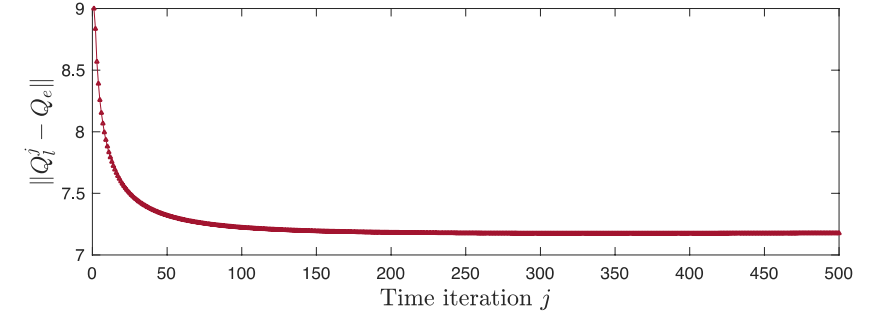
$$\begin{aligned} & V_l^{ji}(x_l(t+T)) - V_l^{ji}(x_l(t)) \\ & - \int_t^{t+T} 2 \left((u_l^{j(i+1)})^T R_l (u_l - u_l^{ji}) \right. \\ & \quad \left. - (v_l^{j(i+1)})^T S_l (v_l - v_l^{ji}) \right) d\tau \\ & = \int_t^{t+T} \left(-q^T(x_l) Q_l^j q(x_l) \right. \\ & \quad \left. - (u_l^{ji})^T R_l u_l^{ji} + (v_l^{ji})^T S_l v_l^{ji} \right) d\tau. \end{aligned}$$

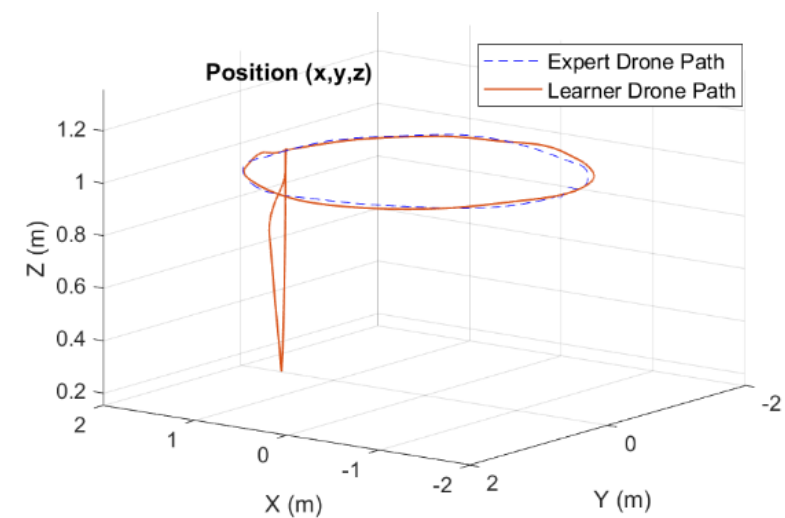
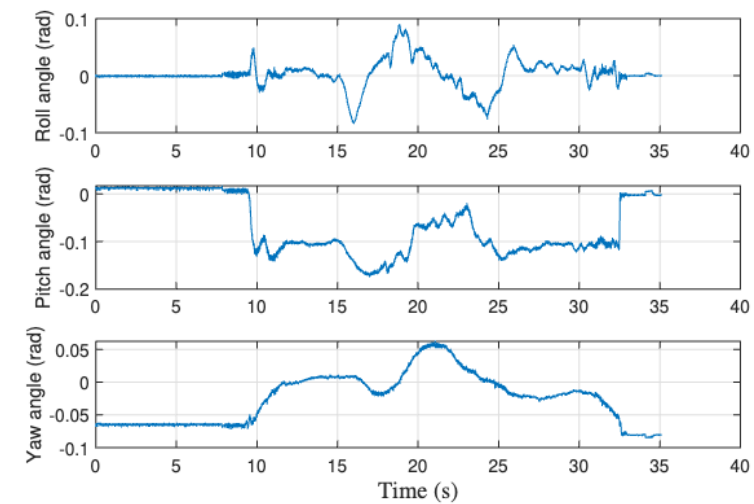
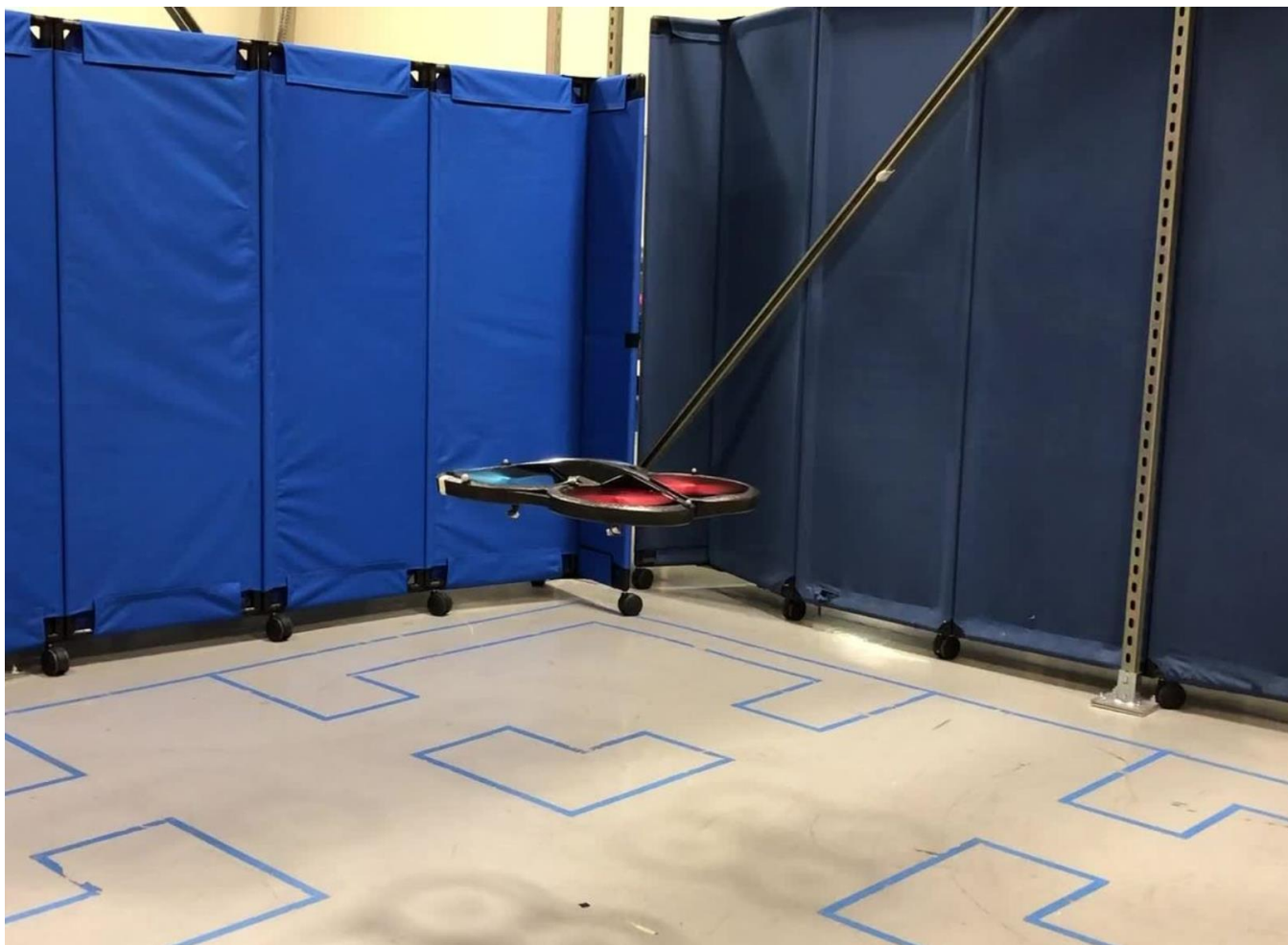
5: Stop if $\|V_l^{ji} - V_l^{j(i-1)}\| \leq \varepsilon_1$, then set $V_l^j = V_l^{ji}$, $u_l^j = u_l^{ji}$ and $v_l^j = v_l^{ji}$. Otherwise, set $i \leftarrow i + 1$ and go to Step 4.

6: **State-penalty weight Q_l^{j+1} update using the expert's demonstration u_e^***

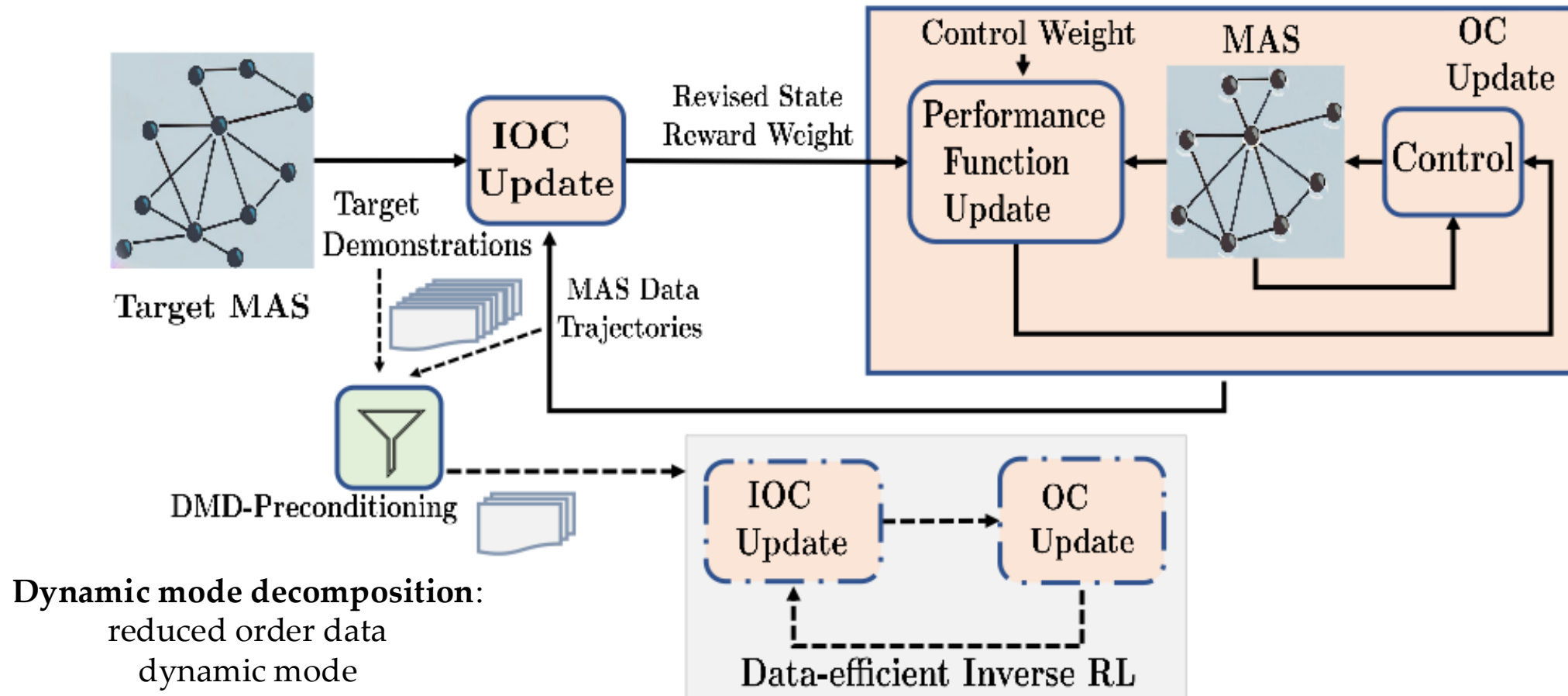
$$\begin{aligned} & \int_t^{t+T} q^T(x_l) Q_l^{j+1} q(x_l) d\tau \\ & = \int_t^{t+T} \left(u_e^{*T} R_l u_e^* - 2u_e^{*T} R_l u_l^j + (v_l^j)^T S_l v_l^j \right) d\tau \\ & \quad - V_l^j(x_l(t+T)) + V_l^j(x_l(t)). \end{aligned}$$

7: **Stop if $\|Q_l^{j+1} - Q_l^j\| \leq \varepsilon_2$.** Otherwise, set $u_l^{(j+1)0} = u_l^j$ and $j \leftarrow j + 1$, then go to Step 3.





Accelerated Inverse RL in Large-Scale Systems



■ Reduced-Order Inverse RL Algorithms

Algorithm 1 Data-efficient Model-free Scalable Inverse Reinforcement Learning Algorithm

- 1 **Start:** Given l , set $h = 0$, and select a small threshold e .
For MAS in (2), select initial $\bar{Q}_r^0 \succeq 0$ and $\bar{R}_r \succ 0$.
- 2 **Data collection:** For $h = 0, 1, 2, \dots, l-1$, collect target MAS data trajectories $(\bar{x}_e[k], \bar{u}_e[k])$ and $(\bar{x}[k], \bar{u}[k])$.
- 3 **Build (D, Λ) & (D_e, Λ_e) :**
 - Take data matrices (ψ_1, ψ_2) and (ψ_{e1}, ψ_{e2}) as inputs.
 - Augment: $\psi_2 = q\psi_1$ and $\psi_{e2} = q\psi_{e1}$.
 - Compute SVD: $(U, S, V) \leftarrow \psi_1, (U_e, S_e, V_e) \leftarrow \psi_{e1}$.
 - Truncate: $(\tilde{U}, \tilde{S}, \tilde{V}) \leftarrow \psi_1, (\tilde{U}_e, \tilde{S}_e, \tilde{V}_e) \leftarrow \psi_{e1}$.
 - Output: Identify (D, Λ) and (D_e, Λ_e) .
- 4 **Preconditioning of datasets:** Compute $(\rho_{rk}, \phi_{rk}, \varrho_r, \Phi_r)$ and $(\theta_{rk}, \omega_{rk}, \Theta_r, \Omega_r)$ given in (39a), (C.5), and (C.8).
- 5 **Update cost matrix and feedback gain:** Compute L_r^{h+1} using (40a) and update K_r^{h+1} as in (40b).
- 6 **Update state reward weight:** Compute Q_r^{h+1} using (40c).
- 7 **Stop if** $\|K_r^{h+1} - K_r^h\| \leq e$. Otherwise, set $h \leftarrow h + 1$ and go to Step 4.
- 8 **Retrieve:**

$$K^{h+1} = DK_r^{h+1}, \quad (41)$$

$$\bar{Q}^{h+1} = D^\dagger \bar{Q}_r^{h+1} D. \quad (42)$$

Order Reduction

Inverse RL

$$\rho_{rk} = \begin{bmatrix} \rho_{xx}(\Lambda_e \bar{\otimes} \Lambda_e) \in \mathbb{R}^{1 \times (r(r+1)/2)} \\ \rho_{xu}(I_{pN} \otimes \Lambda_e) \in \mathbb{R}^{1 \times rpN} \\ \rho_{uu} \in \mathbb{R}^{1 \times (pN(pN+1)/2)} \end{bmatrix}^\top$$

$$\theta_{rk} = \theta_k(\Lambda \bar{\otimes} \Lambda)$$

Inverse RL Control of 10 Linear Aircraft Dynamics

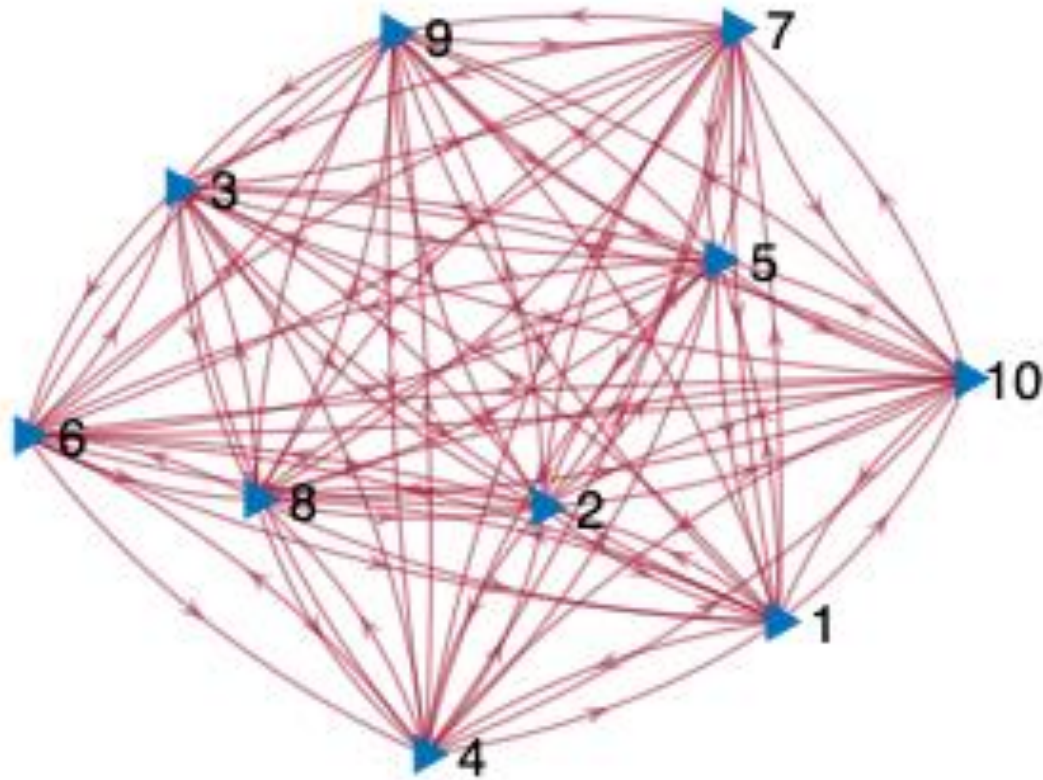
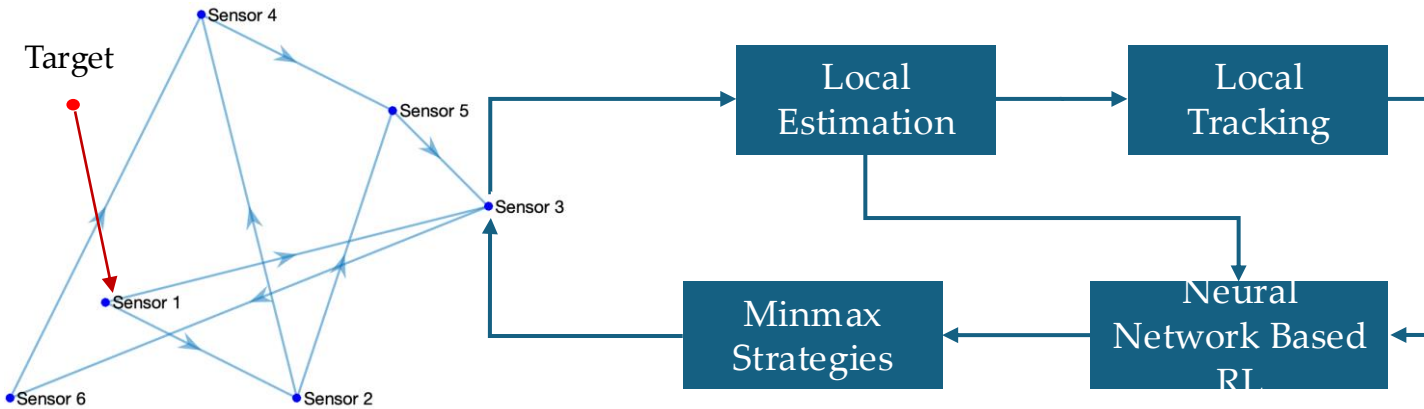


TABLE I
COMPUTATIONAL TIME FOR REWARD-SHAPING USING [20] VS
ALGORITHM 1

Scenario	Spatio-temporal dimensions of state	Dimension of operators	Computational time (<i>Seconds</i>)
Without Pre-conditioning [20]	$\mathbb{R}^{30 \times 1800}$	$\rho_k \in \mathbb{R}^{1 \times 820}$ $\theta_k \in \mathbb{R}^{1 \times 465}$	545.347
With Preconditioning	$\mathbb{R}^3 \times 1800$	$\rho_{rk} \in \mathbb{R}^{1 \times 91}$ $\theta_{rk} \in \mathbb{R}^{1 \times 6}$	1.07

500× speedup

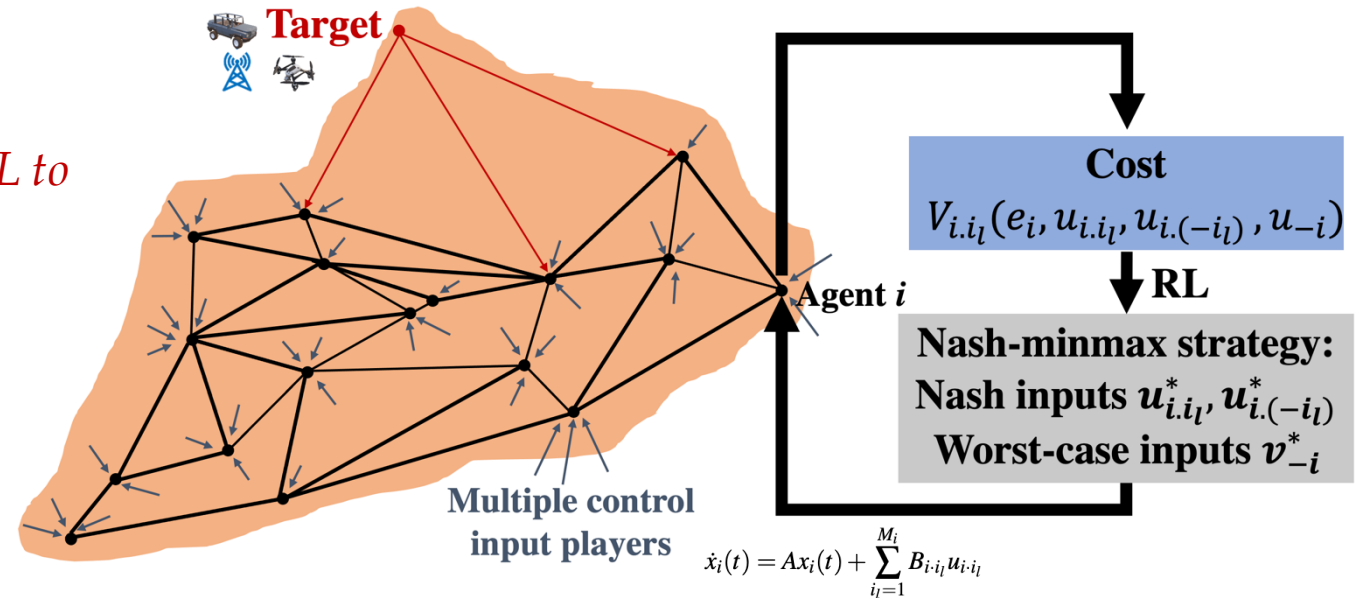
Online Data-Driven RL Control



Lian, B., Lewis, F. L., Hewer, G. A., Estabridis, K., & Chai, T. Online learning of minmax solutions for distributed estimation and tracking control of sensor networks in graphical games. *IEEE Transactions on Control of Network Systems*, 9(4), 1923-1936, 2022.

Model multiagent graphical games to find adaptive optimal control strategy of each agent and design RL to solve HJB equations online.

Lian, B., Xue, W., Lewis, F.L., and Davoudi, A. Nash-minmax strategy for multiplayer multiagent graphical games with reinforcement learning, *IEEE Transactions on Control of Network Systems*, Apr. 2024.





AUBURN
UNIVERSITY



Thank you!

If any question, please contact
bz10098@auburn.edu